كلية الهندسة بشبرا
FACULTY OF ENGINEERING AT SHOUBRA

# High Performance Computing

## Histogram of Images

Made By:

| BN | الإسم |
|---|---|
| 35 | مؤمن محمد احمد حسن |
| 40 | يوسف طه سعد |
| 18 | عبد الله خالد |

Fourth Year Computer Engineering Mainstream

Academic year : 2025-2026

Table of

# Contents


1 Project Overview

2 System Design

3 Binary Input Format

4 Histogram Method

5 Parallel Processing Strategy

6 Output Structure

8 Containerized Cluster Environment

9 Implementation Highlights

10 Experimental Setup

11 Results

12 Performance Discussion

## Project Overview

This project implements a distributed application for computing image histograms from binary matrix files using MPI in the C programming language, with Docker used to create a reproducible multi-container execution environment. The work was designed around a practical High Performance Computing problem: how to process a large collection of image-like data files more efficiently by spreading the work across several parallel processes.

At its core, the application takes a directory of binary files, reads each file as a matrix-based image, computes a quantized three-dimensional histogram from its pixel values, and stores the results in JSONL format. To support controlled testing, a separate matrix generator was also developed. This utility creates synthetic datasets of configurable size, which made it possible to evaluate the behavior of the parallel program under different workloads.

The project is a good example of file-level data parallelism. Each input file can be processed independently, which makes the problem naturally suited to MPI. The program therefore focuses less on inter-process communication during computation and more on efficient workload distribution, local processing, and final result aggregation.

# System Design

The overall system is built from two executable programs.

## Matrix generator

The first program, **generate_matrices**, is a data generation tool. It creates binary matrix files with user-defined values for:

- output directory,
- number of files,
- rows,
- columns,
- number of channels,
- random seed.

This utility was important because it gave full control over the input size and ensured that experiments could be repeated consistently.

## MPI histogram processor

The second program, **mpi_histogram**, is the main application. It is responsible for:

- discovering input files,
- dividing the work across MPI ranks,
- computing a histogram for each assigned file,
- producing per-rank JSONL outputs,
- and merging the rank outputs into a final file.

The source code is organized into separate modules, each with a clear responsibility:

- **fs_utils.c** for filesystem operations,
- **string_list.c** for dynamic storage of file paths,
- **matrix_hist.c** for histogram computation,
- **output_utils.c** for writing and merging output files,
- **mpi_histogram.c** for MPI coordination,
- **generate_matrices.c** for synthetic dataset creation.

This modular structure makes the project easier to understand and gives it a cleaner, more maintainable design than a single-file implementation would.

## Binary Input Format

Each image is stored as a custom binary file. The format begins with a small header:

- **rows** as **uint32_t,**
- **cols** as **uint32_t**,
- **channels** as **uint8_t**.

After this header, the file contains the raw pixel payload as unsigned 8-bit values. The implementation supports two image types:

- grayscale matrices with 1 channel,
- RGB matrices with 3 channels.

In the experiments, all generated files used RGB format with dimensions:

- **4096 × 4096 × 3**

This means each file is large enough to make performance differences observable, especially when processing many files together.

## Histogram Method

The histogram is computed by grouping pixel intensities into bins. Instead of using all 256 possible values directly, the code quantizes each channel according to a configurable **window_size**.
The number of bins per channel is calculated as:

$$bins = \frac{256 + window_size - 1}{window_size}$$

For the experiments, the chosen value was:

- **window_size = 16**

This gives:

- **bins = 16**

Because the histogram is computed over red, green, and blue channels together, the final histogram is three-dimensional. Its total number of entries is therefore:

$$16^3 = 4096$$

For every pixel:

- its red, green, and blue values are extracted,
- each value is mapped to a bin index,
- those indices are combined into a single histogram position,
- and the corresponding count is incremented.

If the input file is grayscale, the same value is assigned to red, green, and blue so that the program can still use the same 3D histogram representation.
One of the more practical implementation choices is that histogram computation is done using chunk-based file reading. Rather than loading an entire large image into memory, the program reads the pixel payload in blocks and updates the histogram gradually. This reduces memory usage and makes the solution more suitable for large datasets.

## Parallel Processing Strategy

The parallelization model is straightforward and well matched to the problem.
After MPI starts, each process obtains:

- its rank,
- and the total number of processes.

The program then identifies all **.bin** files in the input directory and sorts them. Files are divided among ranks using a round-robin scheme:

- rank 0 handles file indices **0, size, 2×size, ...**
- rank 1 handles **1, 1+size, 1+2×size, ...**
- and so on.

This strategy is simple, but in this project it is effective because the input files are almost identical in size and structure. That means each file represents roughly the same amount of work, so the round-robin assignment naturally gives good load balance.

Each rank then performs the following sequence:

1. open its own output part file,
2. process its assigned input files one by one,
3. compute a histogram for each file,
4. write one JSONL record per file,
5. count how many files it processed.

After local processing is complete:

- the local file counts are collected at rank 0 using **MPI_Gather**,
- all ranks synchronize,
- rank 0 merges the part files,
- and rank 0 prints a final timing and workload summary.

This is a good example of an embarrassingly parallel workload. Most of the heavy work is local to each rank, and only minimal coordination is needed at the end.

## Output Structure

The output is written in JSONL format, where each line represents the histogram of one processed image.

Each record contains:

- the file name,
- the number of pixels,
- the number of bins per channel,
- the full histogram array.

This output format was a sensible choice for a technical project because it is:

- human-readable,
- easy to parse,
- and flexible for later analysis.

In addition to the result file, the program prints a compact summary in JSON format. This summary includes:

- total number of files found,
- number of files processed by each rank,
- output file path,
- elapsed execution time.

This summary is especially helpful during experimentation, because it allows both correctness and workload distribution to be checked immediately after each run.

## Containerized Cluster Environment

To make the system easier to run and evaluate, the project uses Docker and Docker Compose to create a small MPI cluster made of three containers:

- **master**
- **worker1**
- **worker2**

All containers are built from the same Docker image and share the same working directory through a mounted volume. The image includes:

- build tools,
- OpenMPI,
- SSH client and server.

SSH is configured so that **mpirun** can launch processes across containers. The entrypoint script starts the SSH daemon automatically, and a separate run script defines the hostfile and executes the MPI program with a selected number of processes.

This setup is valuable from a practical HPC viewpoint. It allows distributed behavior to be tested on a single machine while still following a realistic multi-node workflow. It also improves reproducibility, since the software environment is defined in the container image rather than depending on local machine configuration.

# Implementation Highlights

Several implementation choices make the project more robust and easier to work with.

**Recursive file discovery**

The input directory is scanned recursively, so files can be placed in nested directories without changing the program logic. Only files ending in **.bin** are considered valid inputs.

**Dynamic path storage**

The **StringList** structure stores discovered file paths and expands dynamically as needed. This avoids hardcoded limits and allows the program to scale to larger datasets.

**Portable filesystem support**

The filesystem utilities contain separate handling for Windows and POSIX environments. Even though the project was executed in Linux containers, this shows an effort to make the code more portable.

**Controlled memory use**

Histogram computation uses fixed-size chunk reads instead of full-file loading. This is a practical choice for large images and contributes to more stable memory usage.

**Per-rank output files**

Each rank writes to its own part file. This avoids concurrent writes to a shared output file, which would otherwise require extra synchronization and could reduce performance.

**Clear build process**

The **Makefile** keeps compilation simple and direct, and the Docker build process compiles the project inside a consistent environment. For a university project, this is a strong point because it makes the work easier to reproduce and demonstrate.

# Experimental Setup

Three synthetic datasets were generated for evaluation. All files used the same dimensions and number of channels, while the total number of files changed to produce different overall dataset sizes.

| Dataset | Number of Files | Matrix Dimensions | Channels | Approximate Size |
|---|---|---|---|---|
| Small | 22 | 4096 × 4096 | 3 | ~1 GB |
| Medium | 64 | 4096 × 4096 | 3 | ~3 GB |
| Large22 | 214 | 4096 × 4096 | 3 | ~10 GB |

The datasets were generated using the following commands:

- Small dataset: 22 files
- Medium dataset: 64 files
- Large dataset: 214 files

All experiments used:

- RGB matrices,
- fixed matrix size of **4096 × 4096**,
- histogram **window_size = 16**.

Two main experimental studies were carried out:

1. keeping the number of workers fixed at 6 while increasing dataset size,
2. keeping the dataset fixed at approximately 10 GB while increasing the number of workers.

# Results

## Scaling with dataset size at 6 workers

| Dataset | Number of Files | Workers | Processed per Rank | Time (s) |
|---|---|---|---|---|
| Small | 22 | 6 | [4, 4, 4, 4, 3, 3] | 1.044939 |
| Medium | 64 | 6 | [11, 11, 11, 11, 10, 10] | 3.027274 |
| Large22 | 214 | 6 | [36, 36, 36, 36, 35, 35] | 9.657675 |

**Interpretation**

The first experiment shows that runtime increases as the dataset becomes larger, which is expected. What is more important is that the increase is fairly smooth and proportional to the amount of data being processed. This suggests that the implementation remains stable as workload grows.

The workload split is also balanced in each run. Since the files were uniform in size, the round-robin assignment produced an even distribution with at most a one-file difference between ranks. This is a positive sign that the chosen scheduling strategy was appropriate for the test conditions.

## Scaling with worker count on the 10 GB dataset

| Workers | Processed per Rank | Time (s) | Speedup | Efficiency |
|---|---|---|---|---|
| 1 | [214] | 36.296985 | 1.00 | 100.00% |
| 2 | [107, 107] | 19.301639 | 1.88 | 94.03% |
| 4 | [54, 54, 53, 53] | 12.610616 | 2.88 | 71.97% |
| 6 | [36, 36, 36, 36, 35, 35] | 10.621132 | 3.42 | 57.00% |

## Results

Speedup was calculated relative to the 1-worker case using:

$$Speedup = \frac{T_1}{T_P}$$

and efficiency was calculated as:

$$Efficiency = \frac{Speedup}{p} \times 100$$

**Interpretation**

The second experiment shows a clear benefit from parallel execution. Moving from 1 worker to 2 workers almost halves the runtime, which indicates very good initial scaling. At 4 and 6 workers, the runtime continues to decrease, although the gains become smaller.

This is a normal outcome in parallel systems. In practice, speedup is rarely perfectly linear because some parts of execution remain serial or introduce overhead. In this project, likely sources of overhead include:

- MPI startup and synchronization,
- file system access from multiple processes,
- writing separate part files,
- final merging at rank 0,
- and container-level networking and coordination.

Even so, the performance improvement is substantial. Reducing the processing time from about 36.3 seconds to about 10.6 seconds on the largest dataset shows that the application benefits meaningfully from distributed execution.

# Performance Discussion

The results show that the project meets its main HPC objective: using parallelism to reduce total execution time for a large data-processing task.

A few observations stand out:

**Good balance for equal-sized files**
Because the synthetic datasets contain files with the same dimensions and structure, the round-robin assignment produces an almost ideal distribution. This is reflected directly in the processed_per_rank values.

**Strong improvement from parallelism**
The drop from 36.3 seconds with 1 worker to 10.6 seconds with 6 workers is significant. This confirms that the problem is highly parallelizable and that the MPI approach is well suited to it.

**Efficiency decreases as worker count rises**
Although performance improves with more workers, efficiency drops as more processes are added. This is typical in HPC. The useful work is distributed, but the fixed overheads are not. As the number of workers increases, those overheads make up a larger fraction of total runtime.

**I/O remains an important factor**
This application is not purely CPU-bound. It reads large files from disk and writes large outputs. As a result, storage and file access behavior have a direct effect on performance. This explains why scaling is good but not ideal.